გაკვეთილი 27 mod 3 (3.5)

გაკვეთილი 2 — CSS სელექტორები და DOM ელემენტების შიგთავსის შეცვლა

1. თეორიული მასალა

1.1. წინა გაკვეთილის გამეორება

წინა გაკვეთილზე ვისწავლეთ, როგორ მოვძებნოთ HTML ელემენტები JavaScript-ის გამოყენებით.

ამისთვის გამოვიყენეთ სამი მეთოდი:

```
document.getElementById("title");

document.getElementsByClassName("text");

document.getElementsByTagName("p");
```

ამ მეთოდების საშუალებით შეგვეძლო ელემენტების მოძიება `id`-ის, კლასისა და ტეგის მიხედვით.

დღეს ვისწავლით უფრო მოქნილ მეთოდებს, რომლებითაც შეგვიძლია ნებისმიერი CSS სელექტორის გამოყენება.

გარდა ამისა, ვისწავლით მოძიებული ელემენტების ტექსტის, HTML შიგთავსისა და ატრიბუტების შეცვლას.

2. `querySelector()` — ელემენტის მოძიება CSS სელექტორით

`querySelector()` არის JavaScript-ის მეთოდი, რომელიც გამოიყენება HTML დოკუმენტში პირველი შესაბამისი ელემენტის მოსაძებნად.

მეთოდში შეგვიძლია გამოვიყენოთ CSS სელექტორები.

მაგალითად:

- `#title` — ელემენტის მოძიება ID-ით.
- `.text` — ელემენტის მოძიება კლასით.
- `p` — ელემენტის მოძიება ტეგით.

- `div p` — `div`-ის შიგნით არსებული პირველი `p` ელემენტის მოძიება.

მაგალითი 1 — ელემენტის მოძიება ID-ით

HTML:

```
<h1 id="title">Hello World</h1>
<p>Welcome to my website</p>
```

JavaScript:

```
let heading = document.querySelector("#title");

console.log(heading);
```

განმარტება:

- `document` — წარმოადგენს HTML დოკუმენტს.
- `querySelector()` — ეძებს შესაბამის ელემენტს.
- `#title` — ნიშნავს, რომ ვეძებთ `id="title"` მქონე ელემენტს.
- `heading` — ცვლადია, რომელშიც ინახება მოძიებული ელემენტი.

კონსოლში გამოჩნდება:

```
<h1 id="title">Hello World</h1>
```

მაგალითი 2 — ელემენტის მოძიება კლასით

HTML:

```
<p class="description">First paragraph</p>
<p class="description">Second paragraph</p>
<p class="description">Third paragraph</p>
```

JavaScript:

```
let paragraph = document.querySelector(".description");

console.log(paragraph);
```

შედეგი:

```
<p class="description">First paragraph</p>
```

მიუხედავად იმისა, რომ სამი ელემენტი გვაქვს ერთი და იმავე კლასით, `querySelector()` მხოლოდ პირველ შესაბამის ელემენტს აბრუნებს.

მაგალითი 3 — ელემენტის მოძიება ტეგით

HTML:

```
<h2>First Title</h2>
<h2>Second Title</h2>
```

JavaScript:

```
let title = document.querySelector("h2");

console.log(title);
```

შედეგი:

```
<h2>First Title</h2>
```

მნიშვნელოვანი: თუ შესაბამისი ელემენტი არ არსებობს, `querySelector()` აბრუნებს `null` მნიშვნელობას.

3. `querySelectorAll()` — რამდენიმე ელემენტის მოძიება

`querySelectorAll()` გამოიყენება ყველა იმ HTML ელემენტის მოსაძებნად, რომელიც შეესაბამება მითითებულ CSS სელექტორს.

მთავარი განსხვავება:

- `querySelector()` აბრუნებს პირველ შესაბამის ელემენტს.
- `querySelectorAll()` აბრუნებს ყველა შესაბამის ელემენტს.

მაგალითი 1 — ყველა პარაგრაფის მოძიება

HTML:

```
<p class="text">JavaScript</p>
<p class="text">Python</p>
```

```
<p class="text">Java</p>
<p class="text">C++</p>
```

JavaScript:

```
let paragraphs = document.querySelectorAll(".text");

console.log(paragraphs);
```

მეთოდი აბრუნებს `NodeList` კოლექციას.

კოლექციის ელემენტებთან მუშაობა შეგვიძლია ინდექსების დახმარებით:

```
console.log(paragraphs[0]);
console.log(paragraphs[1]);
console.log(paragraphs[2]);
```

ელემენტების რაოდენობა:

```
console.log(paragraphs.length);
```

შედეგი:

```
4
```

3.1. `forEach()` მეთოდი

`querySelectorAll()`-ის მიერ დაბრუნებულ `NodeList`-ზე შეგვიძლია გამოვიყენოთ `forEach()`.

ეს მეთოდი გამოიყენება კოლექციის თითოეულ ელემენტზე გარკვეული მოქმედების შესასრულებლად.

მაგალითი:

```
let paragraphs = document.querySelectorAll(".text");

paragraphs.forEach(function(paragraph) {
    console.log(paragraph);
});
```

განმარტება:

1. `querySelectorAll(".text")` პოულობს ყველა ელემენტს, რომელსაც აქვს `text` კლასი.
2. `forEach()` გადაუვლის თითოეულ ელემენტს.
3. `paragraph` წარმოადგენს მიმდინარე ელემენტს.
4. `console.log()` გამოიტანს ამ ელემენტს.

იგივე შეგვიძლია დავწეროთ Arrow Function-ით:

```
paragraphs.forEach((paragraph) => {
    console.log(paragraph);
});
```

შეგვიძლია გამოვიტანოთ ელემენტების ინდექსებიც:

```
paragraphs.forEach((paragraph, index) => {
    console.log(index, paragraph);
});
```

3.2. `NodeList` და `HTMLCollection`

წინა გაკვეთილზე გავეცანით `HTMLCollection`-ს.

ახლა მნიშვნელოვანია გავიგოთ განსხვავება მასსა და `NodeList`-ს შორის.

თვისება	HTMLCollection	NodeList
ელემენტებთან ინდექსით წვდომა	კი	კი
`length` თვისება	კი	კი
`forEach()` მეთოდი	პირდაპირ არა	კი
კოლექცია ავტომატურად განახლდება?	ამ კურსში განხილული HTMLCollection-ები — კი	`querySelectorAll()`-ის NodeList — არა

მაგალითად:

```
let items = document.getElementsByClassName("item");

let elements = document.querySelectorAll(".item");
```

პირველი ცვლადი ინახავს `HTMLCollection`-ს, ხოლო მეორე — `NodeList`-ს.

4. HTML ელემენტების ტექსტის შეცვლა

აქამდე JavaScript-ის დახმარებით მხოლოდ ვპოულობდით ელემენტებს.

ახლა ვისწავლით, როგორ შევცვალოთ მათი შიგთავსი.

ამისთვის გამოიყენება სამი ძირითადი თვისება:

- `textContent`
- `innerText`
- `innerHTML`

4.1. `textContent`

`textContent` გამოიყენება HTML ელემენტის ტექსტური შიგთავსის მისაღებად ან შესაცვლელად.

მაგალითი 1 — ტექსტის მიღება

HTML:

```
<h1 id="title">Hello World</h1>
```

JavaScript:

```
let heading = document.querySelector("#title");

console.log(heading.textContent);
```

შედეგი:

```
Hello World
```

მაგალითი 2 — ტექსტის შეცვლა

```
let heading = document.querySelector("#title");

heading.textContent = "Welcome to JavaScript!";
```

ახლა ვებგვერდზე გამოჩნდება:

ცვლილებამდე

Hello World

ცვლილების შემდეგ

Welcome to JavaScript!

HTML ფაილი არ შეგვიცვლია, თუმცა JavaScript-მა ბრაუზერში არსებული ტექსტი განაახლა.

მაგალითი 3 — რამდენიმე პარაგრაფის შეცვლა

HTML:

```
<p class="text">First</p>
<p class="text">Second</p>
<p class="text">Third</p>
```

JavaScript:

```
let paragraphs = document.querySelectorAll(".text");

paragraphs.forEach((paragraph) => {
    paragraph.textContent = "Updated Text";
});
```

შედეგად სამივე პარაგრაფის ტექსტი გახდება `Updated Text`.

მაგალითი 4 — ინდექსების გამოყენება

```
let paragraphs = document.querySelectorAll(".text");

paragraphs.forEach((paragraph, index) => {
    paragraph.textContent = "Paragraph " + (index + 1);
});
```

შედეგი:

```
Paragraph 1
Paragraph 2
Paragraph 3
```

4.2. `innerText`

`innerText` ასევე გამოიყენება ელემენტის ტექსტთან სამუშაოდ.

ის ძირითადად ითვალისწინებს ელემენტის ვიზუალურად გამოსახულ ტექსტს.

მაგალითი:

HTML:

```
<p id="message">Hello Student!</p>
```

JavaScript:

```
let message = document.querySelector("#message");

console.log(message.innerText);

message.innerText = "Welcome to our lesson!";
```

ახლა პარაგრაფში გამოჩნდება:

```
Welcome to our lesson!
```

რა განსხვავებაა `innerText` -სა და `textContent` -ს შორის?

განვიხილოთ მაგალითი:

HTML:

```
<div id="content">
    Hello
    <span style="display: none;">Secret Text</span>
    World
</div>
```

JavaScript:

```
let content = document.querySelector("#content");

console.log(content.textContent);

console.log(content.innerText);
```

`textContent` აბრუნებს ტექსტს, რომელიც ელემენტის შიგნით არსებობს, მათ შორის დამალულ ტექსტსაც.

`innerText` კი ითვალისწინებს იმას, თუ რომელი ტექსტია ვიზუალურად გამოსახული. ამ მაგალითში დამალული `Secret Text` არ იქნება `innerText`-ის შედეგში.

5. `innerHTML` — HTML შიგთავსის შეცვლა

`innerHTML` გამოიყენება ელემენტის შიგნით არსებული HTML კოდის მისაღებად ან შესაცვლელად.

`textContent`-ისგან განსხვავებით, `innerHTML`-ით შეგვიძლია HTML ტეგების დამატებაც.

მაგალითი 1 — ახალი სათაურის დამატება

HTML:

```
<div id="container"></div>
```

JavaScript:

```
let container = document.querySelector("#container");

container.innerHTML = "<h2>Hello Students!</h2>";
```

შედეგი:

Hello Students!

JavaScript-მა `div` ელემენტის შიგნით დაამატა `h2` სათაური.

მაგალითი 2 — რამდენიმე HTML ელემენტის დამატება

HTML:

```
<div id="box"></div>
```

JavaScript:

```
let box = document.querySelector("#box");

box.innerHTML = `
    <h2>My Profile</h2>
```

```
    <p>Name: Giorgi</p>
    <p>Age: 21</p>
    <strong>Web Developer</strong>
`;
```

შედეგი:

My Profile

Name: Giorgi

Age: 21

Web Developer

ამ შემთხვევაში JavaScript-მა ერთდროულად რამდენიმე HTML ელემენტი დაამატა.

5.1. `textContent` და `innerHTML` — მთავარი განსხვავება

HTML:

```
<div id="example"></div>
```

ვარიანტი 1:

```
let example = document.querySelector("#example");

example.textContent = "<h2>Hello World</h2>";
```

შედეგი:

`<h2>Hello World</h2>`

ტეგები გამოჩნდება ჩვეულებრივი ტექსტის სახით.

ვარიანტი 2:

```
example.innerHTML = "<h2>Hello World</h2>";
```

შედეგი:

Hello World

ამ შემთხვევაში ბრაუზერი ქმნის ნამდვილ `h2` ელემენტს.

უსაფრთხოების წესი: მომხმარებლის მიერ შეყვანილი ტექსტი პირდაპირ `innerHTML`-ში არ უნდა ჩავსვათ, რადგან ამან შეიძლება არასასურველი HTML ან JavaScript კოდის შესრულება გამოიწვიოს. ჩვეულებრივი ტექსტისთვის უმჯობესია `textContent`.

6. HTML ატრიბუტებთან მუშაობა

HTML ელემენტებს აქვთ სხვადასხვა ატრიბუტი.

მაგალითად:

```
<img src="image.jpg" alt="My Image">

<a href="https://google.com">Google</a>

<input type="text" value="Hello">
```

JavaScript-ით შეგვიძლია ამ ატრიბუტების წაკითხვა და შეცვლა.

6.1. `src` ატრიბუტის შეცვლა

HTML:

```
<img id="photo" src="photo1.jpg" alt="Photo">
```

JavaScript:

```
let image = document.querySelector("#photo");

image.src = "photo2.jpg";
```

ამ კოდის შესრულების შემდეგ სურათის წყარო შეიცვლება.

ფაილი `photo2.jpg` უნდა არსებობდეს მითითებულ მისამართზე.

6.2. `href` ატრიბუტის შეცვლა

HTML:

```
<a id="link" href="https://google.com">
    Visit Website
</a>
```

JavaScript:

```
let link = document.querySelector("#link");

link.href = "https://github.com";
```

ახლა ბმულზე გადასვლისას გაიხსნება GitHub-ის ვებგვერდი.

6.3. `value` თვისება

`value` გამოიყენება ფორმის ელემენტებთან, მაგალითად `input` -თან მუშაობისას.

HTML:

```
<input id="username" type="text" value="Giorgi">
```

JavaScript:

```
let input = document.querySelector("#username");

console.log(input.value);
```

შედეგი:

```
Giorgi
```

შეგვიძლია მნიშვნელობის შეცვლაც:

```
input.value = "Nika";
```

ამის შემდეგ ტექსტის ველში გამოჩნდება `Nika` .

7. `getAttribute()` და `setAttribute()`

ეს მეთოდები გამოიყენება HTML ატრიბუტების წასაკითხად და შესაცვლელად.

7.1. `getAttribute()`

`getAttribute()` აბრუნებს მითითებული ატრიბუტის მნიშვნელობას.

HTML:

```
<a id="website" href="https://google.com">
    Google
```

```
</a>
```

JavaScript:

```
let website = document.querySelector("#website");

let address = website.getAttribute("href");

console.log(address);
```

შედეგი:

```
https://google.com
```

7.2. `setAttribute()`

`setAttribute()` გამოიყენება ატრიბუტის მნიშვნელობის შესაცვლელად ან ახალი ატრიბუტის დასამატებლად.

HTML:

```
<img id="my-image" src="old.jpg" alt="Old image">
```

JavaScript:

```
let image = document.querySelector("#my-image");

image.setAttribute("src", "new.jpg");

image.setAttribute("alt", "New image");
```

შედეგად სურათს ექნება ახალი `src` და `alt` ატრიბუტები.

დამატებითი მაგალითი

```
<a id="link">My Website</a>
```

```
let link = document.querySelector("#link");

link.setAttribute("href", "https://github.com");
```

```
link.setAttribute("target", "_blank");
```

ახლა ბმულს ექნება `href` ატრიბუტი და მისი დაჭერისას გვერდი გაიხსნება ახალ ჩანართში.

8. სრული პრაქტიკული მაგალითი — პროფილის გვერდი

შევქმნათ ვებგვერდი, სადაც JavaScript-ის დახმარებით შეიცვლება პროფილის ინფორმაცია.

HTML — `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Profile Page</title>
    <script src="script.js" defer></script>
</head>
<body>

    <h1 id="title">My Profile</h1>

    <h2 class="name">Unknown User</h2>

    <p class="description">No information</p>
    <p class="description">No information</p>

    <img
        id="profile-image"
        src="old.jpg"
        alt="Profile Photo"
        width="200"
    >

    <div id="skills"></div>

    <a id="profile-link" href="#">
        My GitHub
    </a>

</body>
</html>
```

JavaScript — `script.js`

```
// 1. სათაურის შეცვლა
let title = document.querySelector("#title");

title.textContent = "Welcome to My Profile";

// 2. სახელის შეცვლა
let name = document.querySelector(".name");

name.textContent = "Giorgi";

// 3. პარაგრაფების შეცვლა
let descriptions = document.querySelectorAll(".description");

descriptions[0].textContent = "I am a Web Developer";

descriptions[1].textContent = "I love JavaScript";

// 4. პროფილის სურათის შეცვლა
let image = document.querySelector("#profile-image");

image.setAttribute("src", "profile.jpg");

// 5. HTML შიგთავსის დამატება
let skills = document.querySelector("#skills");

skills.innerHTML = `
    <h3>My Skills</h3>
    <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
    </ul>
`;

// 6. ბმულის შეცვლა
let link = document.querySelector("#profile-link");

link.href = "https://github.com";
link.setAttribute("target", "_blank");
```

რას აკეთებს პროგრამა?

1. ცვლის გვერდის მთავარ სათაურს.
2. ცვლის მომხმარებლის სახელს.
3. ანახლებს ორივე პარაგრაფის ტექსტს.

4. ცვლის პროფილის სურათს.
5. ქმნის უნარების HTML სიას.
6. ცვლის GitHub-ის ბმულის მისამართს.

9. პრაქტიკული სავარჯიშოები

მოსწავლეებმა უნდა შექმნან HTML და JavaScript ფაილები და შეასრულონ დავალებები.

სავარჯიშო 1 — სათაურებისა და ტექსტების შეცვლა

შექმენით HTML გვერდი, რომელშიც იქნება:

- ერთი `h1`, რომელსაც ექნება `id="main-title"`.
- ორი `h2` ელემენტი.
- სამი პარაგრაფი `class="description"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ მთავარი სათაური `querySelector()`-ით.
2. შეცვალეთ სათაურის ტექსტი `textContent`-ით.
3. მოძებნეთ ორივე `h2` ელემენტი `querySelectorAll()`-ით.
4. შეცვალეთ ორივე სათაურის ტექსტი.
5. `forEach()`-ის დახმარებით შეცვალეთ ყველა პარაგრაფის ტექსტი.
6. კონსოლში გამოიტანეთ, რამდენი პარაგრაფია გვერდზე.

სავარჯიშო 2 — პროგრამირების ენების სია

შექმენით HTML გვერდი, სადაც იქნება სათაური და 5 `li` ელემენტი.

ყველა `li` ელემენტს ჰქონდეს `class="language"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ ყველა ელემენტი `querySelectorAll()`-ით.
2. `forEach()`-ის დახმარებით მიანიჭეთ ელემენტებს შემდეგი ტექსტები: `HTML`, `CSS`, `JavaScript`, `TypeScript`, `Python`.
3. გამოიტანეთ ყველა ელემენტის ტექსტი კონსოლში.
4. დაითვალეთ ელემენტების რაოდენობა.
5. ცალკე შეცვალეთ მესამე ელემენტის ტექსტი `React`-ით.

სავარჯიშო 3 — სურათებისა და ბმულების შეცვლა

შექმენით HTML გვერდი, რომელშიც იქნება:

- ორი `img` ელემენტი.
- ორი `a` ელემენტი.
- ერთი `input` ელემენტი.

JavaScript-ის გამოყენებით:

1. ორივე სურათს შეუცვალეთ `src` ატრიბუტი.
2. `setAttribute()`-ის გამოყენებით ორივე სურათს შეუცვალეთ `alt` ატრიბუტი.
3. ორივე ბმულს შეუცვალეთ `href` ატრიბუტი.
4. ორივე ბმულს დაამატეთ `target="_blank"`.
5. `input` ელემენტში JavaScript-ით ჩაწერეთ თქვენი სახელი.
6. კონსოლში გამოიტანეთ `input`-ის მნიშვნელობა.

სავარჯიშო 4 — სტუდენტის პროფილი

შექმენით სტუდენტის პროფილის ვებგვერდი.

HTML გვერდზე უნდა იყოს:

- მთავარი სათაური.
- სტუდენტის სახელი და გვარი.
- ასაკის პარაგრაფი.
- უნივერსიტეტის დასახელება.
- პროფილის სურათი.
- ბმული უნივერსიტეტის ვებგვერდზე.
- ცარიელი `div`, რომელსაც ექნება `id="courses"`.

JavaScript-ის გამოყენებით:

1. შეცვალეთ მთავარი სათაურის ტექსტი.
2. შეცვალეთ სტუდენტის სახელი და გვარი.
3. განაახლეთ ასაკისა და უნივერსიტეტის ტექსტები.
4. შეცვალეთ პროფილის სურათი.
5. `innerHTML`-ის გამოყენებით `courses` ელემენტში დაამატეთ სათაური და მინიმუმ 4 სასწავლო კურსის სია.
6. შეცვალეთ უნივერსიტეტის ბმულის მისამართი.
7. ბმულს დაამატეთ `target="_blank"` ატრიბუტი.
8. კონსოლში გამოიტანეთ მთავარი სათაურის ტექსტი და ბმულის `href` ატრიბუტი.